

数算期末 Cheat Sheet

目录

1. 排序算法	2. 全排列	3. 栈的应用
4. 表达式求值与转换	5. 建树与Huffman	6. 并查集
7. 最小生成树 Prim/Kruskal	8. 补图BFS与完全图MST	9. 拓扑排序 Kahn/DFS
10. 欧拉路径 Hierholzer	11. 关键路径 AOE网	12. 最短路径算法
13. 最小斯坦纳树	14. 强连通分量 Kosaraju/Tarjan	15. 无向图连通与判环
16. 二分查找	17. 位运算技巧	18. 线段树
19. 树状数组 BIT	20. LCA 倍增法	21. 二叉树概念与遍历
22. Trie 字典树	23. KMP算法	24. Manacher回文串
25. TSP 旅行商	26. 经典题：招生工作	27. 经典回溯：骑士周游/八皇后
28. 数值格式化	29. Python考试陷阱	30. 链表
31. 树形DP	32. 括号嵌套树	

1. 排序算法

归并排序（可求逆序对数） <pre>def Merge(a,start,mid,end): tmp,l,r=[],start,mid+1 while l<=mid and r<=end: if a[l]<=a[r]: tmp.append(a[l]); l+=1 else: tmp.append(a[r]); r+=1 tmp.extend(a[l:mid+1]) tmp.extend(a[r:end+1]) for i in range(start,end+1): a[i]=tmp[i-start] def MergeSort(a,start,end): if start==end: return mid=(start+end)//2 MergeSort(a,start,mid) MergeSort(a,mid+1,end) Merge(a,start,mid,end)</pre>	快速排序 <pre>def quicksort(arr, left, right): if left < right: pos = partition(arr, left, right) quicksort(arr, left, pos - 1) quicksort(arr, pos + 1, right) def partition(arr, left, right): i, j, pivot = left, right-1, arr[right] while i <= j: while i<=right and arr[i]<pivot: i+=1 while j>=left and arr[j]>=pivot: j-=1 if i < j: arr[i],arr[j] = arr[j],arr[i] if arr[i] > pivot: arr[i],arr[right] = arr[right],arr[i] return i</pre>
---	--

2. 全排列

递归交换法（原地，无visited数组） <pre>def permutations(arr): res = [] def backtrack(start): if start == len(arr): res.append(arr[:]) return for i in range(start, len(arr)): arr[start],arr[i] = arr[i],arr[start] backtrack(start+1) arr[start],arr[i] = arr[i],arr[start] backtrack(0) return res</pre>	下一个字典序 next_permutation O(n) 从右找第一个升序对 arr[i]<arr[i+1]；再从右找 arr[j]>arr[i]，交换；反转 i+1 到末尾。 <pre>def next_permutation(arr): n=len(arr); i=n-2 while i>=0 and arr[i]>=arr[i+1]: i-=1 if i<0: return False j=n-1 while arr[j]<=arr[i]: j-=1 arr[i],arr[j]=arr[j],arr[i] left,right=i+1,n-1 while left<right: arr[left],arr[right]=arr[right],arr[left] left+=1; right-=1 return True</pre> 标准库： <code>from itertools import permutations</code>
---	---

3. 栈的应用

单调栈（奶牛排队：求最长子序列，左界严格最小，右界严格最大）

```
N,res=int(input()),0
hi=[int(input()) for _ in range(N)]
left,right=[-1]*N,[N]*N
stack1,stack2=[],[]
for i in range(N-1,-1,-1):
    while stack1 and hi[stack1[-1]]>hi[i]:
        stack1.pop()
    if stack1: right[i]=stack1[-1]
    stack1.append(i)
for i in range(N):
    while stack2 and hi[stack2[-1]]<hi[i]:
        stack2.pop()
    if stack2: left[i]=stack2[-1]
    stack2.append(i)
for i in range(N):
    for j in range(right[i]-1,i,-1):
        if left[j]<i:
            res=max(j-i+1,res); break
print(res)
```

合法出栈序列（M30637）

双指针+模拟栈：对seq中每个字符c，从x中push直到栈顶==c再pop。

```
import sys

def is_valid_pop(x, seq):
    if len(seq)!=len(x) or set(seq)!=set(x):
        return False
    stack, i = [], 0
    for c in seq:
        while not stack or stack[-1]!=c:
            if i>=len(x): return False
            stack.append(x[i]); i+=1
        stack.pop()
    return True

data = sys.stdin.read().split('\n')
x = data[0].strip()
for line in data[1:]:
    line = line.strip()
    if not line: continue
    print("YES" if is_valid_pop(x,line) else "NO")
```

每个字符最多入栈/出栈一次，单次O(n)。

完美交易窗口（T30102，单调栈）

题意：N秒股价数组h，找最长连续子段[i..j]满足：①h[i]是区间最小值（完美买入）②h[j]是区间最大值（完美卖出）③h[j]>h[i]（盈利）④中间价格不等于h[i]也不等于h[j]（严格完美）。输出最长窗口秒数。N≤10⁶。

样例：[3,1,2,5,4,6,1,2,4,3] → 5（窗口[1,2,5,4,6]，买1卖6）

思路：维护单调递增栈，栈中每个元素存 [买入点下标, 该区域内的最大值下标]。从左到右扫描：- 当前价格 ≤ 栈顶 → 栈顶作为“唯一最小值”的统治结束，弹出并结算 - 弹出时用 right_max_idx 接力棒传递右侧最大值位置 - 若接力棒有效（right_max_idx ≠ r），则 [buy_idx, right_max_idx] 是候选窗口 - 末尾加哨兵 -1 强制清空栈

```
import sys

def solve():
    input_data = sys.stdin.read().split()
    n = int(input_data[0])
    prices = [int(x) for x in input_data[1:n+1]]
    prices.append(-1) # 哨兵，强制弹出所有元素

    buy_stack = [] # [买入点下标, 区域最大值下标]
    max_window_len = 0

    for r in range(n + 1):
        current_price = prices[r]
        right_max_idx = r # 接力棒：右侧最大值位置

        # 当前价格 ≤ 栈顶 → 栈顶买入点的统治结束
        while buy_stack and prices[buy_stack[-1][0]] ≥ current_price:
            buy_idx, sub_max_idx = buy_stack.pop()

            # 接力棒有效 → 存在右侧比买入点高的卖出点
            if right_max_idx ≠ r:
                current_len = right_max_idx - buy_idx + 1
                if current_len > max_window_len:
                    max_window_len = current_len

            # 更新接力棒：取更大的那个最大值
            if prices[sub_max_idx] ≥ prices[right_max_idx]:
                right_max_idx = sub_max_idx

        buy_stack.append([r, right_max_idx])

    print(max_window_len)

solve()
```

复杂度：每个元素最多入栈/出栈一次，O(n)。**两个if顺序不能换：**先用旧的right_max_idx算长度，再更新它；第一次弹栈时right_max_idx=r故跳过（紧邻当前点的买入点右侧还没有更高卖出点）。

Unix路径简化（栈的经典应用）

题意：给一个Unix风格的绝对路径字符串（如 /a/./b/../../c/），简化为最简路径（/c）。规则：/ 分隔，. 当前目录（忽略），.. 上一级（弹栈），连续 // 当一个 /。

样例：/home//foo/../../bar → /home/bar，/../ → /，/a/./b/../../c/ → /c

关键思路：路径从左到右就是"从根目录往下走"，天然匹配栈的语义。.. = 弹栈，普通名 = 压栈。**千万不要从右往左处理**—— .. 需要"回头消掉前面的目录"会导致嵌套循环和边界爆炸。

```
def simplify_path(path):
    parts = path.split('/')
    stack = []
    for part in parts:
        if part == '' or part == '.':
            continue # 空串(连续//)和.都跳过
        elif part == '..':
            if stack:
                stack.pop() # 回上一级
        else:
            stack.append(part) # 进入子目录
    return '/' + '/'.join(stack)

# 使用
print(simplify_path(input().strip()))
```

单调栈/队列进阶应用

最大全0矩形面积 (OJ27205, 按行累积高度+单调递增栈)

```
m, n = map(int, input().split())
forest = [list(map(int, input().split())) for _ in range(m)]
height = [0]*n
ans = 0
for i in range(m):
    height = [h+1 if v==0 else 0 for h, v in zip(height, forest[i])]
    stack = []
    for j in range(n+1):
        cur = height[j] if j < n else -1
        while stack and height[stack[-1]] > cur:
            h = height[stack.pop()]
            w = j - stack[-1] - 1 if stack else j
            ans = max(ans, h*w)
        stack.append(j)
    print(ans)
```

滑动窗口最大值 (LC239, 单调递减双端队列存索引)

```
from collections import deque
def max_sliding_window(nums, k):
    dq = deque() # 存索引, 对应值单调递减
    res = []
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] < x:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k: # 队首滑出窗口
            dq.popleft()
        if i >= k - 1:
            res.append(nums[dq[0]])
    return res
```

队首始终是窗口最大值。每个元素进/出队各一次, O(n)。

4. 表达式求值与转换

后缀表达式求值

```
def cal(a,b,op):
    if op=="+":return a+b
    if op=="-":return a-b
    if op=="*":return a*b
    if op=="/":return a/b

from collections import deque
n,ops=int(input()),("+", "-", "*", "/")
raw=[deque(map(str,input().split()))
      for _ in range(n)]
for deq in raw:
    tmp=deque()
    while len(deq) >= 1:
        if deq[0] not in ops:
            tmp.append(float(deq.popleft()))
        else:
            b=tmp.pop(); a=tmp.pop()
            op=deq.popleft()
            deq.appendleft(cal(a,b,op))
    print('{:.2f}'.format(tmp[0]))
```

中缀转后缀 (Shunting Yard)

操作数→输出；（→栈；运算符→弹出优先级≥的再入栈；）→弹到左括号。

```
def infix_to_postfix(expr):
    prec={'+':1, '-':1, '*':2, '/':2}
    out,stack,buf=[],[],[]
    def flush():
        if buf:out.append(''.join(buf));buf.clear()
    for c in expr:
        if c.isdigit() or c=='.': buf.append(c)
        elif c=='(':
            flush(); stack.append(c)
        elif c==')':
            flush()
            while stack and stack[-1]!='(':
                out.append(stack.pop())
            stack.pop()
        elif c in prec:
            flush()
            while (stack and
                   prec.get(stack[-1],0) >= prec[c]):
                out.append(stack.pop())
            stack.append(c)
    flush()
    while stack: out.append(stack.pop())
    return ''.join(out)
```

5. 建树与Huffman

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
```

Huffman编码树: 构造扩充二叉树使 $\sum W_i \times L_i$ 最小。每次取两个最小权节点合并。

```
import heapq
class HuffmanTreeNode:
    def __init__(self, weight, char=None):
        self.weight=weight; self.char=char; self.left=None; self.right=None
    def __lt__(self, other): return self.weight<other.weight

def BuildHuffmanTree(characters):
    heap=[HuffmanTreeNode(w,c) for c,w in characters.items()]
    heapq.heapify(heap)
    while len(heap)>1:
        left,right=heapq.heappop(heap),heapq.heappop(heap)
        merged=HuffmanTreeNode(left.weight+right.weight)
        merged.left,merged.right=left,right
        heapq.heappush(heap,merged)
    return heapq.heappop(heap)

def enpaths(root):
    paths={}
    def traverse(node,path):
        if node.char: paths[(node.char,node.weight)]=path
        else: traverse(node.left,path+1); traverse(node.right,path+1)
    traverse(root,0); return paths

def min_weighted_path(paths):
    return sum(t[1]*p for t,p in paths.items())
```

唯一性规则 (确保编码树唯一) : 比较节点时, 权值小的算小; 权值相同则字符集最小字符小的算小; 合并时小节点作左子 (边为0), 大节点作右子 (边为1) 。

```
import heapq
class Node:
    def __init__(self, val='', weight=0):
        self.val = val; self.weight = weight
        self.left = None; self.right = None
    def __lt__(self, other):
        if self.weight == other.weight:
            return self.val[0] < other.val[0] # 权同比最小字符
        return self.weight < other.weight

def build(freq):
    # freq: {字符: 权值}
    heap = [Node(c, w) for c, w in freq.items()]
    heapq.heapify(heap)
    while len(heap) > 1:
        a, b = heapq.heappop(heap), heapq.heappop(heap)
        merged = Node(''.join(sorted(a.val + b.val)), a.weight + b.weight)
        merged.left, merged.right = a, b
        heapq.heappush(heap, merged)
    return heap[0]

def encode(root, s):
    # 字符串 → 01串
    ans, node = '', root
    for ch in s:
        node = root
        while node.left or node.right:
            if ch in node.left.val: ans += '0'; node = node.left
            else: ans += '1'; node = node.right
    return ans

def decode(root, bits):
    # 01串 → 字符串
    ans, node = '', root
    for ch in bits:
        node = node.left if ch == '0' else node.right
        if not node.left and not node.right:
            ans += node.val; node = root
    return ans
```

6. 并查集

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
    def find(self, x):
        if self.parent[x] != x: self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return
        if self.rank[rx] > self.rank[ry]: self.parent[ry] = rx
        elif self.rank[rx] < self.rank[ry]: self.parent[rx] = ry
        else: self.parent[ry] = rx; self.rank[rx] += 1
```

发现它抓住它 (2倍并查集, 敌对关系)

每案件两个节点: 本身 + 对立。

```
def solve():
    n,m=map(int,input().split())
    uf=UnionFind(2*n)
    for _ in range(m):
        op,a,b=input().split()
        a,b=int(a)-1,int(b)-1
        if op=="D":
            uf.union(a,b+n); uf.union(a+n,b)
        else:
            if (uf.find(a)==uf.find(b) or
                uf.find(a+n)==uf.find(b+n)):
                print("In the same gang.")
            elif (uf.find(a)==uf.find(b+n) or
                  uf.find(a+n)==uf.find(b)):
                print("In different gangs.")
            else: print("Not sure yet.")
```

食物链 (3倍并查集)

x =同类, x+n =x吃的, x+2n =吃x的 - 同类→union(x,y), union(x+n,y+n), union(x+2n,y+2n) - x吃y→union(x+n,y), union(x,y+2n), union(x+2n,y+n) - 冲突: find(x)==find(y的其他角色) 即假话

食物链 (BFS染色法, 带权图思路)

思路: A吃B吃C吃A, 是一个长度为3的循环关系。给每个动物染色0/1/2, 表示它在食物链中的"相对角色"。用BFS/DFS传播: - "x与y同类"→ 颜色必须相同: color[y] = color[x] - "x吃y"→ 颜色差1: color[y] = (color[x]+1) % 3

若传播时发现矛盾 (已染色的节点颜色不符) → 假话。

```
from collections import deque

def solve():
    n, k = map(int, input().split())
    # adj[x] = [(y, relation), ...]
    # relation=0: 同类, relation=1: x吃y
    adj = [[] for _ in range(n + 1)]
    color = [-1] * (n + 1) # -1=未染色, 0/1/2=角色
    fake = 0

    for _ in range(k):
        op, x, y = map(int, input().split())
        if x > n or y > n: # 编号超范围
            fake += 1; continue
        if op == 1: # x与y同类
            adj[x].append((y, 0))
            adj[y].append((x, 0))
        else: # x吃y
            if x == y: # 自己吃自己
                fake += 1; continue
            adj[x].append((y, 1)) # x→y 差1
            adj[y].append((x, 2)) # y→x 差2 (反向)

    # BFS 染色: 逐个连通分量
    def check_component(start):
        """对start所在连通分量染色, 返回矛盾的边数"""
        bad = 0
        color[start] = 0
        queue = deque([start])
        while queue:
            u = queue.popleft()
            for v, rel in adj[u]:
                expected = (color[u] + rel) % 3
                if color[v] == -1:
                    color[v] = expected
                    queue.append(v)
                elif color[v] != expected:
                    bad += 1
        return bad

    # 按输入顺序逐条检查更精确, 但按连通分量染色
    # 是简化版: 只能检测全局矛盾
    for i in range(1, n + 1):
        if color[i] == -1:
            fake += check_component(i)

    # 注意: 染色法检测的是全局矛盾数,
    # 和原题"按输入顺序逐条判断假话"的计数方式略有不同。
    # 原题精确计数推荐用并查集; 染色法适合判"是否存在矛盾"。
    print(fake)

solve()
```

并查集 vs 染色法对比

方法	优势	适用
3倍并查集	支持在线逐条判断, 精确计数假话	原题要求按顺序计数
BFS染色	直观易理解, 容易扩展到k种关系	判断整体是否有矛盾

偏序集极大/极小元 (并查集+有向边标记)

```
def count_extremes(n, equiv, less_than):
    uf=UnionFind(n)
    for a,b in equiv: uf.union(a,b)
    not_max,not_min=set(),set()
    for a,b in less_than:
        ra,rb=uf.find(a),uf.find(b)
        if ra!=rb:
            not_max.add(ra); not_min.add(rb)
    roots={uf.find(i) for i in range(1,n+1)}
    return len(roots-not_max),len(roots-not_min)
```

变形：一般有向图先Tarjan求SCC缩点再套用。

猫猫搭积木（T30921，并查集+崩塌/启发式合并）

题意：n块积木，每次选两块x,y合并所在堆。若合并后堆大小≥s，该堆崩塌变成零散的s堆（每块独立）。q次操作后输出当前堆数。

样例：n=5, q=10, s=3 → 输出序列 4 3 5 4 3 2 5 4 3 2

启发式合并（直观，维护elements列表）

```
import sys

def solve():
    data = list(map(int, sys.stdin.read().split()))
    it = iter(data)
    n, q, s = next(it), next(it), next(it)
    parent = list(range(n + 1))
    size = [1] * (n + 1)
    elements = [[i] for i in range(n + 1)]
    piles = n
    out = []
    for _ in range(q):
        u, v = next(it), next(it)
        ru, rv = parent[u], parent[v]
        if ru != rv:
            su, sv = size[ru], size[rv]
            if su + sv >= s: # 崩塌
                piles += su + sv - 2
                for x in elements[ru]:
                    parent[x]=x; size[x]=1
                    elements[x]=[x]
                for x in elements[rv]:
                    parent[x]=x; size[x]=1
                    elements[x]=[x]
            else: # 启发式合并
                piles -= 1
                if su < sv:
                    for x in elements[ru]:
                        parent[x]=rv
                        elements[rv].extend(elements[ru])
                        size[rv]+=su; elements[ru]=[]
                else:
                    for x in elements[rv]:
                        parent[x]=ru
                        elements[ru].extend(elements[rv])
                        size[ru]+=sv; elements[rv]=[]
            out.append(str(piles))
    print('\n'.join(out))
solve()
```

懒标记法（崩塌时不逐个拆，打“销毁”标记，访问时按需分配新ID）

```
class UnionFind:
    def __init__(self, n, q, s):
        self.s = s
        self.destroyed = n + 2*q
        total = n + 2*q + 1
        self.parent = list(range(total))
        self.rank = [0]*total
        self.num = [1]*total
        self.count = n
        self.fakeid = list(range(n))
        self.realid = list(range(n + 2*q))
        self.safeplace = n
    def _find(self, x):
        if self.parent[x]!=x:
            self.parent[x]=self._find(self.parent[x])
        return self.parent[x]
    def find(self, x):
        if self._find(x)==self.destroyed:
            rid = self.realid[x]
            self.fakeid[rid]=self.safeplace
            self.realid[self.safeplace]=rid
            x = self.safeplace
            self.safeplace += 1
        return self.parent[x]
    def union(self, x, y):
        x,y = self.fakeid[x], self.fakeid[y]
        rx,ry = self.find(x), self.find(y)
        if rx!=ry:
            if self.rank[rx]<self.rank[ry]:
                rx,ry=ry,rx
            self.parent[ry]=rx
            self.num[rx]+=self.num[ry]
            if self.rank[rx]==self.rank[ry]:
                self.rank[rx]+=1
            self.count -= 1
            if self.num[rx]>=self.s:
                self.count+=(self.num[rx]-1)
                self.parent[rx]=self.destroyed

n,q,s = map(int,input().split())
uf = UnionFind(n,q,s)
for _ in range(q):
    a,b = map(int,input().split())
    uf.union(a-1,b-1)
print(uf.count)
```

方法	思路	适用
启发式合并	崩塌时逐个拆散，小堆并入大堆	直观，s较大时效率高
懒标记	崩塌只打标记，访问时才分配新ID	适合频繁崩塌场景

Prim (堆优化) 每次取开销最小的边扩展

```
from heapq import heappop, heappush
def prim(matrix):
    ans, N = 0, len(matrix)
    pq, visited = [(0, 0)], [False] * N
    while pq:
        c, cur = heappop(pq)
        if visited[cur]: continue
        visited[cur] = True; ans += c
        for i in range(N):
            if not visited[i] and matrix[cur][i] != 0:
                heappush(pq, (matrix[cur][i], i))
    return ans
```

注意: 一个节点可能多次存在于pq中, `if visited: continue` 是关键。

Kruskal (并查集) 按权排序, 加不成环的边

```
def kruskal(graph):
    n = len(graph)
    res, edges, dsj = [], [], DisJointSet(n)
    for i in range(n):
        for j in range(i+1, n):
            if graph[i][j] != 0:
                edges.append((i, j, graph[i][j]))
    for u, v, w in sorted(edges, key=lambda x: x[2]):
        if dsj.find(u) != dsj.find(v):
            dsj.union(u, v); res.append((u, v, w))
    return res
# print(sum(e[2] for e in kruskal(graph)))
```

能写Prim建议写Prim。

8. 补图BFS与完全图MST

核心技巧: 补图BFS (链表+临时移除)

问题: 在n个点的完全图中, 给出m条"特殊边", 求补图(非特殊边构成的图)的连通分量。朴素做法要枚举 $O(n^2)$ 条边, 太慢。

核心思想: 用双向链表维护"未分配的顶点集"。处理顶点u时: 1. **临时移除** u在原图中的邻居(特殊边的另一端)——它们不是补图邻居 2. 链表中**剩下的所有顶点**都是u的补图邻居 → 全部加入当前连通分量, **永久删除** 3. **恢复**临时移除的邻居(它们可能是其他顶点的补图邻居)

复杂度: 每个顶点最多永久删除1次 $O(n)$, 每条特殊边的端点最多被临时移除/恢复两次 $O(m)$, 总计 **$O(n+m)$** 。

```
# 补图BFS模板: 返回每个顶点所属的连通分量编号 comp[1..n]
from collections import deque

def complement_bfs(n, adj):
    """adj[u] = u的原图邻居列表(特殊边)"""
    nxt = [i + 1 for i in range(n + 2)]
    pre = [i - 1 for i in range(n + 2)]

    def remove(v):
        nxt[pre[v]] = nxt[v]
        pre[nxt[v]] = pre[v]

    def insert_front(v):
        nxt[v] = nxt[0]
        pre[nxt[0]] = v
        nxt[0] = v
        pre[v] = 0

    comp = [0] * (n + 1)
    cid = 0
    for start in range(1, n + 1):
        if comp[start]: continue
        cid += 1
        comp[start] = cid
        remove(start)
        queue = deque([start])
        while queue:
            u = queue.popleft()
            # 第1步: 临时移除u的原图邻居
            temp = []
            for v in adj[u]:
                if comp[v] == 0: # 还在链表中
                    nxt[pre[v]] = nxt[v] # 临时删除
                    pre[nxt[v]] = pre[v]
                    temp.append(v)
            # 第2步: 链表剩余全是补图邻居→加入当前分量
            v = nxt[0]
            while v <= n:
                nxt_v = nxt[v]
                comp[v] = cid
                remove(v) # 永久删除
                queue.append(v)
                v = nxt_v
            # 第3步: 恢复临时移除的邻居
            for v in temp:
                insert_front(v)
    return comp, cid
```

T30313: 0-W 最小生成树

题意: n个点完全图, m条边权为 $w > 0$ (特殊边), 其余边权为0。求MST权值和。 $n \leq 1e5, m \leq 2e5$ 。

样例: 输入 6 11 + 11条边 → 输出 15

思路: 0权边 = 补图边。补图BFS找出所有0-边连通分量(块内连边免费)。块间只能用特殊边连接 → 对块缩点后按权排序Kruskal。

样例中0权边把{1,2},{4,5,6}各连成块，{3}单独，共3块需2条正权边：3-4:5 + 1-3:10 = **15**。

```
import sys
from collections import deque

def main():
    data = sys.stdin.buffer.read().split(); p = 0
    n, m = int(data[p]), int(data[p+1]); p += 2
    adj = [[] for _ in range(n + 1)]
    edges = []
    for _ in range(m):
        u, v, w = int(data[p]), int(data[p+1]), int(data[p+2]); p += 3
        adj[u].append(v); adj[v].append(u)
        edges.append((w, u, v))

    # 补图BFS求0-边连通分量
    nxt = [i + 1 for i in range(n + 2)]
    pre = [i - 1 for i in range(n + 2)]
    def remove(v):
        nxt[pre[v]] = nxt[v]; pre[nxt[v]] = pre[v]
    def insert_front(v):
        nxt[v] = nxt[0]; pre[nxt[0]] = v; nxt[0] = v; pre[v] = 0

    comp = [0] * (n + 1); cid = 0
    for start in range(1, n + 1):
        if comp[start]: continue
        cid += 1; comp[start] = cid; remove(start)
        queue = deque([start])
        while queue:
            u = queue.popleft()
            temp = []
            for v in adj[u]:
                if comp[v] == 0:
                    nxt[pre[v]] = nxt[v]; pre[nxt[v]] = pre[v]
                    temp.append(v)
            v = nxt[0]
            while v ≤ n:
                nxt_v = nxt[v]; comp[v] = cid; remove(v)
                queue.append(v); v = nxt_v
            for v in temp:
                insert_front(v)

    # 对块缩点后Kruskal
    parent = list(range(cid + 1))
    def find(x):
        while parent[x] ≠ x: parent[x] = parent[parent[x]]; x = parent[x]
        return x
    edges.sort()
    ans = 0
    for w, u, v in edges:
        cu, cv = find(comp[u]), find(comp[v])
        if cu ≠ cv:
            parent[cu] = cv; ans += w
    print(ans)
main()
```

变体：1-W 最小生成树（默认边权1，特殊边权w≥0）

题意： n个点完全图， m条特殊边权为w≥0， **其余边权为1**。求MST权值和。

思路： MST按边权从小到大贪心。将边分成三档处理：

阶段	处理的边	方法
Phase 1	特殊边中 w < 1 的（即 w=0）	排序后Kruskal
Phase 2	默认边（权=1，即补图边）	补图BFS，每次union代价1
Phase 3	特殊边中 w ≥ 1 的	排序后Kruskal

Phase 2详解： 补图BFS找到一个补图连通分量后，其中可能包含k个不同的并查集分量（Phase 1遗留的），需要k-1条权=1的默认边将它们连通，代价为k-1。

```

import sys
from collections import deque

def main():
    data = sys.stdin.buffer.read().split(); p = 0
    n, m = int(data[p]), int(data[p+1]); p += 2
    adj = [[] for _ in range(n + 1)] # 所有特殊边的邻接表
    light, heavy = [], [] # w<1 和 w≥1 的特殊边
    for _ in range(m):
        u, v, w = int(data[p]), int(data[p+1]), int(data[p+2]); p += 3
        adj[u].append(v); adj[v].append(u)
        (light if w < 1 else heavy).append((w, u, v))

    parent = list(range(n + 1))
    rnk = [0] * (n + 1)
    def find(x):
        while parent[x] != x: parent[x] = parent[parent[x]]; x = parent[x]
        return x
    def union(x, y):
        rx, ry = find(x), find(y)
        if rx == ry: return False
        if rnk[rx] < rnk[ry]: rx, ry = ry, rx
        parent[ry] = rx
        if rnk[rx] == rnk[ry]: rnk[rx] += 1
        return True

    ans = 0

    # Phase 1: 特殊边 w < 1 (比默认边便宜)
    light.sort()
    for w, u, v in light:
        if union(u, v): ans += w

    # Phase 2: 默认边 (权=1) via 补图BFS
    nxt = [i + 1 for i in range(n + 2)]
    pre = [i - 1 for i in range(n + 2)]
    def remove(v):
        nxt[pre[v]] = nxt[v]; pre[nxt[v]] = pre[v]
    def insert_front(v):
        nxt[v] = nxt[0]; pre[nxt[0]] = v; nxt[0] = v; pre[v] = 0

    visited = [False] * (n + 1)
    for start in range(1, n + 1):
        if visited[start]: continue
        visited[start] = True; remove(start)
        queue = deque([start])
        while queue:
            u = queue.popleft()
            temp = []
            for v in adj[u]: # 临时移除特殊边邻居
                if not visited[v]:
                    nxt[pre[v]] = nxt[v]; pre[nxt[v]] = pre[v]
                    temp.append(v)
            v = nxt[0]
            while v ≤ n: # 剩余=补图邻居→默认边连接
                nxt_v = nxt[v]
                visited[v] = True; remove(v); queue.append(v)
                if union(start, v): # 跨UF分量才有代价
                    ans += 1
                v = nxt_v
            for v in temp:
                insert_front(v)

    # Phase 3: 特殊边 w ≥ 1 (补图连不通的, 只能用特殊边)
    heavy.sort()
    for w, u, v in heavy:
        if union(u, v): ans += w

    print(ans)
main()

```

两种问题统一理解:

问题	默认边权	Phase 1	Phase 2 (补图BFS)	Phase 3
0-W MST	0	无 (所有特殊边>0)	每次连接代价 0	Kruskal特殊边
1-W MST	1	Kruskal w<1的边	每次连接代价 1	Kruskal w≥1的边

9. 拓扑排序 Kahn/DFS

Kahn算法 (BFS+入度)

不断移除入度为0的顶点。要求按编号顺序输出时用堆。
O(V+E)。

```
from collections import defaultdict
from heapq import heappush,heappop
def Kahn(graph):
    q,ans=[],[]
    in_degree=defaultdict(int)
    for lst in graph.values():
        for v in lst: in_degree[v]+=1
    for v in graph:
        if v not in in_degree or in_degree[v]==0:
            heappush(q,v)
    while q:
        v=heappop(q); ans.append('v'+str(v))
        for nb in graph[v]:
            in_degree[nb]-=1
            if in_degree[nb]==0:
                heappush(q,nb)
    return ans
```

DFS求拓扑序 (带环检测)

按结束时间递减排列即拓扑序。三色标记：白0未访问，灰1在递归栈，黑2完毕。遇灰→有环。

```
def dfs_topo_sort(graph):
    visited={v:0 for v in graph}
    finish_order=[]
    def dfs_visit(u):
        visited[u]=1
        for v in graph.get(u,[]):
            if visited[v]==0:
                if not dfs_visit(v): return False
            elif visited[v]==1:
                return False # 发现环
        visited[u]=2
        finish_order.append(u)
        return True
    for u in graph:
        if visited[u]==0:
            if not dfs_visit(u):
                raise ValueError("图中存在环")
    return finish_order[::-1]
```

选择：按编号顺序→Kahn(堆)；只需任一拓扑序或判环→DFS。

神经网络模拟 (29740, 拓扑排序应用)

题意：n个神经元构成有向图。输入层 (入度=0) 有初始激活值C，非输入层初始C=0。信号沿边传播： $C_i = (\sum W_{ji} \cdot C_j) - U_i$ (仅上游Cj>0的才传播)。输出出度=0且最终C>0的神经元。若图有环则输出NULL。
关键细节： - 输入层的C**不减偏置**，非输入层入度减为0时才减偏置 - 重复边(i,j)权重**累加** - 读入顺序：先状态后偏置 (states, bias = ...) ⚠️
别读反

```

from collections import defaultdict, deque

n, p = map(int, input().split())
states = [0] * (n + 1)
bias = [0] * (n + 1)
for i in range(1, n + 1):
    states[i], bias[i] = map(int, input().split()) # 注意顺序!

edges = defaultdict(int)
in_deg = [0] * (n + 1)
out_deg = [0] * (n + 1)
for _ in range(p):
    a, b, w = map(int, input().split())
    edges[(a, b)] += w # 重复边权累加

for u, v in edges:
    in_deg[v] += 1
    out_deg[u] += 1

# Kahn拓扑排序 + 信号传播
queue = deque()
cnt = 0
for i in range(1, n + 1):
    if in_deg[i] == 0:
        queue.append(i)
        cnt += 1

while queue:
    u = queue.popleft()
    for v in range(1, n + 1): # 遍历u的所有出边
        if (u, v) not in edges:
            continue
        in_deg[v] -= 1
        if states[u] > 0: # 仅激活神经元传播信号
            states[v] += edges[(u, v)] * states[u]
        if in_deg[v] == 0:
            states[v] -= bias[v] # 非输入层: 入度归0时减偏置
            queue.append(v)
            cnt += 1

# 输出
ans = [(i, states[i]) for i in range(1, n + 1)
        if out_deg[i] == 0 and states[i] > 0]

if cnt != n or not ans: # 有环 或 无激活输出
    print("NULL")
else:
    for node, val in sorted(ans):
        print(node, val)

```

二叉的水管（T29702，拓扑排序+完全二叉树）

题意：完全二叉树形水管，每个节点把流量分给左子树、余下给右子树。给出若干“A > B”（A流量>B）的比较关系，重建二叉树并输出**中序遍历**。
矛盾→ Device error. ；信息不足无法唯一确定→ Not determined. 。

核心性质：完全二叉树里节点流量大小**顺序固定**——父>子，且右子树整体>左子树（提示：“左子树根<右子树最小”）。所以**前序按“根→右→左”遍历得到的就是流量从大到小的位置序列**。

做法：①拓扑排序所有比较关系，得到编号的流量降序；②排序中出现环→ Device error. ，某步队列>1（顺序不唯一）→ Not determined. ；③把“根→右→左”的位置序列与拓扑降序——对应，填回数组，中序输出。

```

import sys
from collections import defaultdict, deque

def solve():
    m, n = map(int, sys.stdin.readline().split())
    edges = defaultdict(list)
    indegree = [0]*(m+1)
    for _ in range(n):
        line = sys.stdin.readline().strip()
        if not line: continue
        l, r = line.split('>')          # 按'>'拆，容错空格/无空格
        A, B = int(l), int(r)
        edges[A].append(B); indegree[B] += 1

    # 拓扑排序：判环 + 判唯一
    q = deque(u for u in range(1, m+1) if indegree[u] == 0)
    topo, multiple = [], False
    while q:
        if len(q) > 1: multiple = True   # 同时多个入度0→顺序不唯一
        u = q.popleft(); topo.append(u)
        for v in edges[u]:
            indegree[v] -= 1
            if indegree[v] == 0: q.append(v)

    if len(topo) < m:                    # 有环
        print("Device error."); return
    if multiple:
        print("Not determined."); return

    # 位置序列：前序 根→右→左 = 流量从大到小
    pos_order = []
    def dfs(u):
        if u > m: return
        pos_order.append(u)
        dfs(2*u+1); dfs(2*u)             # 先右后左
    dfs(1)

    assigned = [0]*(m+1)
    for i in range(m):
        assigned[pos_order[i]] = topo[i] # 位置i←流量第i大的编号

    res = []
    def inorder(u):
        if u > m: return
        inorder(2*u); res.append(str(assigned[u])); inorder(2*u+1)
    inorder(1)
    print(" ".join(res))

solve()

```

10. 欧拉路径 Hierholzer

欧拉路径：经过每条边恰好一次。**欧拉回路**：起终点相同的欧拉路径。

存在条件：有向图回路：每点入度=出度；路径：恰一点出-入=1(起)、一点入-出=1(终)。无向图：所有偶度→回路；恰2个奇度点→路径。

递归版（最小字典序，邻接表用最小堆）

```
import heapq, sys
from collections import defaultdict
sys.setrecursionlimit(1<<25)

def hierholzer_min_lex(n, edges):
    graph=defaultdict(list)
    indeg=[0]*(n+1); outdeg=[0]*(n+1)
    for u,v in edges:
        graph[u].append(v)
        outdeg[u]+=1; indeg[v]+=1
    for u in graph: heapq.heapify(graph[u])
    # 检查存在条件
    start=end=-1
    for i in range(1,n+1):
        d=outdeg[i]-indeg[i]
        if d==1:
            if start!=-1: return None
            start=i
        elif d==-1:
            if end!=-1: return None
            end=i
        elif d!=0: return None
    if start==-1 and end==-1:
        start=min((v for v in range(1,n+1)
                    if outdeg[v]>0),default=1)
    elif not(start!=-1 and end!=-1):
        return None
    path=[]
    def dfs(v):
        while graph[v]:
            u=heapq.heappop(graph[v]); dfs(u)
            path.append(v)
    dfs(start); path.reverse()
    return path if len(path)==len(edges)+1 else None
```

迭代版（防递归栈溢出）

```
def hierholzer_iter(start, graph):
    # graph[v] 已 heapify
    stack, path = [start], []
    while stack:
        v = stack[-1]
        if graph[v]:
            stack.append(heapq.heappop(graph[v]))
        else:
            path.append(stack.pop())
    return path[::-1]
```

算法思想：从合法起点DFS，每走过一条边就删除，无路可走时把当前点压入答案栈；最后反转即得欧拉路径。每条边访问一次， $O(V+E)$ 。

11. 关键路径 AOE网

AOE网：边=活动(耗时)，顶点=事件。关键路径=源到汇**最长路径**，slack=0的活动。

算法：①正向拓扑序求 $ve[v]$ （最早）= $\max(ve[u]+w)$ ；②逆拓扑序求 $vl[v]$ （最晚）= $\min(vl[v]-w)$ ；③ $ve[u]=vl[v]-w$ 的边是关键活动。

```
from collections import defaultdict, deque

def critical_path(n, edges):
    G=defaultdict(list); in_deg=[0]*n
    for u,v,w in edges:
        G[u].append((v,w)); in_deg[v]+=1
    deg=in_deg[:]; q=deque(i for i in range(n) if deg[i]==0)
    ve=[0]*n; order=[]
    while q:
        u=q.popleft(); order.append(u)
        for v,w in G[u]:
            ve[v]=max(ve[v],ve[u]+w)
            deg[v]-=1
            if deg[v]==0: q.append(v)
    if len(order)!=n: return -1,[]
    total=max(ve); vl=[total]*n
    for u in reversed(order):
        for v,w in G[u]: vl[u]=min(vl[u],vl[v]-w)
    critical=[(u,v,w) for u,v,w in edges if ve[u]+w==vl[v]]
    return total, critical
```

12. 最短路径算法

Dijkstra通用模板（非负权， $O((V+E)\log V)$ ）

```
import heapq
def dijkstra(n, adj, start):
    dist=[float('inf')]*n; dist[start]=0; pq=[(0,start)]
    while pq:
        d,u=heapq.heappop(pq)
        if d>dist[u]: continue
        for v,w in adj[u]:
            if dist[u]+w<dist[v]:
                dist[v]=dist[u]+w
                heapq.heappush(pq,(dist[v],v))
    return dist
```

路径恢复: 用 `prev[v]` 记录v的前驱, 松弛时更新 `prev[v]=u`, 最后从终点回溯。

```
def dijkstra_path(n, adj, start, end):
    dist=[float('inf')]*n; dist[start]=0
    prev=[-1]*n; pq=[(0,start)]
    while pq:
        d,u=heapq.heappop(pq)
        if d>dist[u]: continue
        for v,w in adj[u]:
            if dist[u]+w<dist[v]:
                dist[v]=dist[u]+w; prev[v]=u
                heapq.heappush(pq,(dist[v],v))
    path=[]; cur=end
    while cur!=start: path.append(cur); cur=prev[cur]
    return path[::-1] if dist[end]<float('inf') else []
```

Dijkstra应用: 道路 (带费用约束)

写法一: 步数剪枝

```
from heapq import heappop,heappush
from collections import defaultdict
K,N,R=int(input()),int(input()),int(input())
graph=defaultdict(list)
for i in range(R):
    S,D,L,T=map(int,input().split())
    graph[S].append((D,L,T))
def Dijkstra(graph):
    q=[]; heappush(q,(0,0,1,0))
    while q:
        l,cost,cur,step=heappop(q)
        if cur==N: return l
        for nxt,nl,nc in graph[cur]:
            if cost+nc<=K and step+1<N:
                heappush(q,(l+nl,cost+nc,nxt,step+1))
    return -1
print(Dijkstra(graph))
```

写法二: 花费剪枝 (推荐)

```
from heapq import heappop,heappush
from collections import defaultdict
K,N,R=int(input()),int(input()),int(input())
graph=defaultdict(list)
for i in range(R):
    S,D,L,T=map(int,input().split())
    graph[S].append((D,L,T))
def Dijkstra(graph):
    q=[]
    min_cost={i:float('inf') for i in range(1,N+1)}
    heappush(q,(0,0,1))
    while q:
        l,cost,cur=heappop(q)
        min_cost[cur]=min(min_cost[cur],cost)
        if cur==N: return l
        for nxt,nl,nc in graph[cur]:
            if cost+nc<=K and nc+cost<min_cost[nxt]:
                heappush(q,(l+nl,cost+nc,nxt))
    return -1
print(Dijkstra(graph))
```

Bellman-Ford (支持负权边, $O(V \cdot E)$)

V-1次松弛所有边, 第V次还能更新一负环。

```
def bellman_ford(n, edges, start):
    dist=[float('inf')]*n; dist[start]=0
    for _ in range(n-1):
        for u,v,w in edges:
            if (dist[u]!=float('inf') and
                dist[u]+w<dist[v]):
                dist[v]=dist[u]+w
    for u,v,w in edges:
        if (dist[u]!=float('inf') and
            dist[u]+w<dist[v]):
            return "存在负权环"
    return dist
```

应用1: 限制最多K次中转的最短路 (LC787)

题意: n个城市、若干航班(u,v,price), 求从src到dst最多经过K次中转的最便宜价格, 不存在返回-1。

思路: 中转K次 = 最多K+1条边。只做K+1轮松弛即可。关键: 每轮必须基于上轮副本 prev=dist[:], 否则同一轮内更新会串扰(一轮内走了多条边)。

```
def findCheapestPrice(n, flights, src, dst, K):
    dist = [float('inf')] * n
    dist[src] = 0
    for _ in range(K + 1):
        prev = dist[:]
        for u, v, w in flights:
            if prev[u] != float('inf') and prev[u] + w < dist[v]:
                dist[v] = prev[u] + w
    return dist[dst] if dist[dst] != float('inf') else -1
```

应用2: 货币兑换套利 (OJ1860)

题意: N种货币、M个兑换点, 每个兑换点双向汇率+手续费。初始V单位货币S, 问能否通过兑换增加S的数量。

思路: 松弛改为 $best[v] = \max(best[v], (best[u]-fee)*rate)$ (求最大收益而非最短路)。做N轮松弛后若仍能更新 → 存在正增益环 → 可套利。

```
import sys
def main():
    data = sys.stdin.read().split(); p = 0
    N, M = int(data[p]), int(data[p+1])
    S, V = int(data[p+2]), float(data[p+3]); p += 4
    edges = []
    for _ in range(M):
        A, B = int(data[p]), int(data[p+1])
        Rab, Cab = float(data[p+2]), float(data[p+3])
        Rba, Cba = float(data[p+4]), float(data[p+5]); p += 6
        edges.append((A, B, Rab, Cab))
        edges.append((B, A, Rba, Cba))
    best = [0.0] * (N + 1)
    best[S] = V
    for it in range(N):
        updated = False
        for u, v, rate, fee in edges:
            if best[u] > fee:
                x = (best[u] - fee) * rate
                if x > best[v] + 1e-12:
                    best[v] = x; updated = True
        if not updated: break
    print("YES" if best[S] > V else "NO")
main()
```

SPFA (BF队列优化, 平均 $O(kE)$, 最坏 $O(VE)$)

只把"刚更新过"的点入队。入队次数 $\geq V \rightarrow$ 负环。

```
from collections import deque
def spfa(graph, V, source):
    dist=[float('inf')]*V; dist[source]=0
    queue=deque([source])
    in_queue=[False]*V; in_queue[source]=True
    count=[0]*V; count[source]=1
    while queue:
        u=queue.popleft(); in_queue[u]=False
        for v,w in graph[u]:
            if dist[u]+w<dist[v]:
                dist[v]=dist[u]+w
                if not in_queue[v]:
                    count[v]+=1
                    if count[v] ≥ V:
                        return "存在负权环"
                if queue and dist[v]<dist[queue[0]]:
                    queue.appendleft(v)
                else: queue.append(v)
            in_queue[v]=True
    return dist
```

Floyd-Warshall (多源最短路, $O(V^3)$): 中间点k必须最外层循环!

```
def floyd_warshall(V, graph):
    dist=[row[:] for row in graph]
    for k in range(V):
        for i in range(V):
            for j in range(V):
                if dist[i][k]+dist[k][j]<dist[i][j]:
                    dist[i][j]=dist[i][k]+dist[k][j]
    return dist
# 负环检测: dist[i][i]<0
```

带路径还原: $nxt[i][j]$ 记录i到j下一个节点, 更新时 $nxt[i][j]=nxt[i][k]$ 。

算法	复杂度	负权	负环检测	适用
Dijkstra	$O((V+E)\log V)$	×	×	单源非负权（首选）
Bellman-Ford	$O(V\cdot E)$	✓	✓	单源有负权、限边数
SPFA	平均 $O(kE)$	✓	✓	单源稀疏图
Floyd	$O(V^3)$	✓	✓	多源 $V\leq 500$

13. 最小斯坦纳树

给无向带权图和 k 个关键点($k\leq 10$)，求包含所有关键点的最小代价子树。 $O(n\cdot 3^k+(n+m)\cdot 2^k\cdot \log n)$ 。

$dp[mask][i]$ =以 i 为根、已连通关键点集 $mask$ 的最小代价。

转移A（同点合并子集）：枚举 $mask$ 非空真子集 sub ， $dp[mask][i]=\min(dp[sub][i]+dp[mask\wedge sub][i])$

转移B（Dijkstra扩展）：对固定 $mask$ 跑多源Dijkstra

```
import heapq

def steiner_tree(n, edges, terminals):
    k=len(terminals)
    adj=[[] for _ in range(n+1)]
    for u,v,w in edges:
        adj[u].append((v,w)); adj[v].append((u,w))
    INF=float('inf')
    dp=[[INF]*(n+1) for _ in range(1<<k)]
    for i,t in enumerate(terminals): dp[1<<i][t]=0
    for mask in range(1,1<<k):
        # A: 枚举子集
        sub=(mask-1)&mask
        while sub>0:
            for i in range(1,n+1):
                if dp[sub][i]+dp[mask^sub][i]<dp[mask][i]:
                    dp[mask][i]=dp[sub][i]+dp[mask^sub][i]
            sub=(sub-1)&mask
        # B: Dijkstra
        pq=[(dp[mask][i],i) for i in range(1,n+1) if dp[mask][i]<INF]
        heapq.heapify(pq)
        while pq:
            d,u=heapq.heappop(pq)
            if d>dp[mask][u]: continue
            for v,w in adj[u]:
                if d+w<dp[mask][v]:
                    dp[mask][v]=d+w
                    heapq.heappush(pq,(dp[mask][v],v))
    return min(dp[(1<<k)-1][1:])
```

子集枚举： $sub=(sub-1)\&mask$ 是经典写法，总复杂度 $O(3^n)$ 。

14. 强连通分量 Kosaraju/Tarjan

Kosaraju (两次DFS+反图)

```
def dfs1(graph,node,visited,stack):
    visited[node]=True
    for nb in graph[node]:
        if not visited[nb]:
            dfs1(graph,nb,visited,stack)
    stack.append(node)

def dfs2(graph,node,visited,comp):
    visited[node]=True; comp.append(node)
    for nb in graph[node]:
        if not visited[nb]:
            dfs2(graph,nb,visited,comp)

def kosaraju(graph):
    stack=[]; visited=[False]*len(graph)
    for node in range(len(graph)):
        if not visited[node]:
            dfs1(graph,node,visited,stack)
    # 转置图
    trans=[] for _ in range(len(graph)):
    for u in range(len(graph)):
        for v in graph[u]: trans[v].append(u)
    visited=[False]*len(graph); sccs=[]
    while stack:
        node=stack.pop()
        if not visited[node]:
            scc=[]
            dfs2(trans,node,visited,scc)
            sccs.append(scc)
    return sccs
```

Tarjan (一次DFS, 常数更小)

idx=访问时间戳, low=能到的最早时间戳。low[v]==idx[v]时v是SCC根。

```
import sys
sys.setrecursionlimit(1<<25)

def tarjan_scc(n, edges):
    graph=[[] for _ in range(n+1)]
    for u,v in edges: graph[u].append(v)
    index=0; idx=[0]*(n+1); low=[0]*(n+1)
    on_stack=[False]*(n+1); stack=[]; sccs=[]
    def dfs(v):
        nonlocal index; index+=1
        idx[v]=low[v]=index
        stack.append(v); on_stack[v]=True
        for w in graph[v]:
            if idx[w]==0:
                dfs(w)
                low[v]=min(low[v],low[w])
            elif on_stack[w]:
                low[v]=min(low[v],idx[w])
        if low[v]==idx[v]:
            comp=[]
            while True:
                w=stack.pop(); on_stack[w]=False
                comp.append(w)
                if w==v: break
            sccs.append(comp)
    for v in range(1,n+1):
        if idx[v]==0: dfs(v)
    return sccs
```

Tarjan返回的SCC顺序是缩点DAG的**逆拓扑序**。

SCC缩点建DAG: 把每个SCC视作超级节点, 原图变成有向无环图, 可在其上跑拓扑/DP。

```
from collections import defaultdict
def build_dag(graph, sccs):
    n = len(graph)
    scc_id = [0] * n
    for i, comp in enumerate(sccs):
        for node in comp: scc_id[node] = i
    dag = defaultdict(set)
    for u in range(n):
        for v in graph[u]:
            if scc_id[u] != scc_id[v]:
                dag[scc_id[u]].add(scc_id[v])
    return dag, scc_id
```

15. 无向图连通与判环

BFS判连通+步数判环

```
from collections import defaultdict,deque
def is_connected(graph,n):
    dq=deque([0]); visited={0}
    while dq:
        cur=dq.popleft()
        for nxt in graph[cur]:
            if nxt not in visited:
                dq.append(nxt); visited.add(nxt)
    return len(visited)==n

def is_loop(graph):
    global_visited=set()
    for vertex in graph:
        if vertex not in global_visited:
            local_visited={}; dq=deque()
            dq.append((vertex,0))
            local_visited[vertex]=0
            global_visited.add(vertex)
            while dq:
                cur,steps=dq.popleft()
                for nxt in graph[cur]:
                    if nxt in local_visited:
                        if local_visited[nxt] >= steps:
                            return True
                    else:
                        dq.append((nxt,steps+1))
                        local_visited[nxt]=steps+1
                        global_visited.add(nxt)
    return False
```

并查集判环 (推荐, 最快 ~O(α(n)·E))

```
class UF:
    def __init__(self,n):
        self.parent=list(range(n))
    def find(self,x):
        if self.parent[x]!=x:
            self.parent[x]=self.find(self.parent[x])
        return self.parent[x]
    def union(self,x,y):
        rx,ry=self.find(x),self.find(y)
        if rx==ry: return False
        self.parent[ry]=rx; return True

def has_cycle_uf(n, edges):
    uf=UF(n)
    for u,v in edges:
        if not uf.union(u,v): return True
    return False
```

DFS+parent判环 (最直观)

```
def has_cycle_dfs(graph, n):
    visited=[False]*n
    def dfs(u, parent):
        visited[u]=True
        for v in graph[u]:
            if not visited[v]:
                if dfs(v,u): return True
            elif v!=parent: return True
        return False
    for u in range(n):
        if not visited[u]:
            if dfs(u,-1): return True
    return False
```

方法	复杂度	优势
BFS+步数	O(V+E)	不递归
并查集	~O(α(n)·E)	最快
DFS+parent	O(V+E)	最直观

16. 二分查找

月度开销：二分答案，check函数判断x作为最大月度开销是否可行。

```
n,m=map(int,input().split())
expend=[int(input()) for i in range(n)]
def check(x):
    nums,s=1,0
    for i in range(n):
        if expend[i]+s>x: s=expend[i]; nums+=1
        else: s+=expend[i]
    return nums>m

lo,hi,res=max(expend),sum(expend)+1,1
while lo<hi:
    mid=(lo+hi)//2
    if check(mid): lo=mid+1
    else: res,hi=mid,mid
print(res)
```

bisect模块 (有序序列O(logN)二分)

```
import bisect
a = [1, 3, 3, 5, 7, 9]
bisect.bisect_left(a, 3) # 1 第一个≥x的位置
bisect.bisect_right(a, 3) # 3 第一个>x的位置
bisect.insort(a, 4) # 插入并保持有序
```

函数	x存在时返回	含义
bisect_left(a,x)	第一个x的下标	第一个≥x
bisect_right(a,x)	最后x的下一位	第一个>x

LIS O(NlogN)：维护"长度为i的LIS末尾最小值"数组d。

```
def length_of_LIS(nums):
    d=[]
    for x in nums:
        i=bisect.bisect_left(d,x) # 严格上升用left
        if i==len(d): d.append(x)
        else: d[i]=x
    return len(d)
```

17. 位运算技巧

运算	写法	说明
与/或/异或/非	& \ ^ ~	~x = -x-1
取第k位	(x>>k)&1	
第k位置1/清0/翻转	x\ (1<<k) / x&~(1<<k) / x^(1<<k)	
lowbit	x & -x	最低位1的值
去最低位1	x & (x-1)	Brian Kernighan
判2的幂	x>0 and x&(x-1)==0	
1的个数	bin(x).count('1') 或 x.bit_count()	
⌊log ₂ (n)⌋	n.bit_length()-1	比int(log2(n))可靠

状态压缩DP常用操作

```
S | (1<<i)      # 加入i
S & ~(1<<i)     # 移除i
(S>>i) & 1      # 检查i
S == (1<<n)-1   # 全集
```

枚举mask所有非空真子集 O(3^n)

```
sub = (mask-1) & mask
while sub > 0:
    # 处理 sub 和 mask^sub
    sub = (sub-1) & mask
```

18. 线段树

区间和+lazy标记 (区间加+区间查询)

```
class SegTree:
    def __init__(self, data):
        self.n=len(data)
        self.tree=[0]*(4*self.n)
        self.lazy=[0]*(4*self.n)
        self.build(1,0,self.n-1,data)
    def build(self,nd,l,r,data):
        if l==r: self.tree[nd]=data[l]; return
        mid=(l+r)//2
        self.build(nd*2,l,mid,data)
        self.build(nd*2+1,mid+1,r,data)
        self.tree[nd]=self.tree[nd*2]+self.tree[nd*2+1]
    def push_down(self,nd,l,r):
        if self.lazy[nd]:
            mid=(l+r)//2; lz=self.lazy[nd]
            self.tree[nd*2]+=lz*(mid-l+1)
            self.lazy[nd*2]+=lz
            self.tree[nd*2+1]+=lz*(r-mid)
            self.lazy[nd*2+1]+=lz
            self.lazy[nd]=0
    def update(self,nd,l,r,ql,qv,val):
        if qr<l or r<ql: return
        if ql<=l and r<=qr:
            self.tree[nd]+=val*(r-l+1)
            self.lazy[nd]+=val; return
        self.push_down(nd,l,r)
        mid=(l+r)//2
        self.update(nd*2,l,mid,ql,qv,val)
        self.update(nd*2+1,mid+1,r,ql,qv,val)
        self.tree[nd]=self.tree[nd*2]+self.tree[nd*2+1]
    def query(self,nd,l,r,ql,qr):
        if qr<l or r<ql: return 0
        if ql<=l and r<=qr: return self.tree[nd]
        self.push_down(nd,l,r)
        mid=(l+r)//2
        return (self.query(nd*2,l,mid,ql,qr)
                +self.query(nd*2+1,mid+1,r,ql,qr))
```

区间最大值 (单点更新版)

```
class SegTreeMax:
    def __init__(self, data):
        self.n=len(data)
        self.tree=[-float('inf')]*(4*self.n)
        self.build(1,0,self.n-1,data)
    def build(self,nd,l,r,data):
        if l==r: self.tree[nd]=data[l]; return
        mid=(l+r)//2
        self.build(nd*2,l,mid,data)
        self.build(nd*2+1,mid+1,r,data)
        self.tree[nd]=max(self.tree[nd*2],
                           self.tree[nd*2+1])
    def update(self,nd,l,r,idx,val):
        if l==r: self.tree[nd]=val; return
        mid=(l+r)//2
        if idx<=mid:
            self.update(nd*2,l,mid,idx,val)
        else:
            self.update(nd*2+1,mid+1,r,idx,val)
        self.tree[nd]=max(self.tree[nd*2],
                           self.tree[nd*2+1])
    def query(self,nd,l,r,ql,qr):
        if qr<l or r<ql: return -float('inf')
        if ql<=l and r<=qr: return self.tree[nd]
        mid=(l+r)//2
        return max(
            self.query(nd*2,l,mid,ql,qr),
            self.query(nd*2+1,mid+1,r,ql,qr))
```

求最小值: max→min, 初值→float('inf').

19. 树状数组 BIT

lowbit(x)=x&-x. 下标从1开始 (lowbit(0)=0会死循环)。

```
n = 10; tree = [0]*(n+1)
def lowbit(x): return x & -x
def update(idx, val):
    while idx<=n: tree[idx]+=val; idx+=lowbit(idx)
def query(idx):
    s=0
    while idx>0: s+=tree[idx]; idx-=lowbit(idx)
    return s
def range_query(l,r): return query(r)-query(l-1)
```

应用1: 求逆序对数

从右往左扫, 查已出现且<a[i]的个数。值域大时先离散化。

```
n=int(input())
a=list(map(int,input().split()))
sorted_a=sorted(set(a))
rank={v:i+1 for i,v in enumerate(sorted_a)}
m=len(sorted_a); tree=[0]*(m+1)
inv=0
for x in reversed(a):
    r=rank[x]
    inv+=query(r-1)
    update(r,1)
print(inv)
```

应用2: 差分BIT (区间修改+单点查询)

维护diff[i]=a[i]-a[i-1], 区间[l,r]加val只需diff[l]+=val, diff[r+1]-=val。

```
n=int(input())
a=list(map(int,input().split()))
tree=[0]*(n+2) # +2防越界
prev=0
for i in range(n):
    update(i+1,a[i]-prev); prev=a[i]
# 区间[l,r]加val:
# update(l,val); update(r+1,-val)
# 查询a[i]: query(i)
```

20. LCA 倍增法

预处理O(NlogN), 查询O(logN)。up[k][v]=v的第2^k个祖先。

```
import sys
from math import log2
sys.setrecursionlimit(200000)

class LCA:
    def __init__(self, n, root, graph):
        self.n=n; self.LOG=max(1,int(log2(n))+1)
        self.depth=[0]*n
        self.up=[[-1]*n for _ in range(self.LOG)]
        self.graph=graph; self.dfs(root,-1,0)
        for k in range(1,self.LOG):
            for v in range(n):
                if self.up[k-1][v]!=-1:
                    self.up[k][v]=self.up[k-1][self.up[k-1][v]]
    def dfs(self, u, parent, d):
        self.up[0][u]=parent; self.depth[u]=d
        for v in self.graph[u]:
            if v!=parent: self.dfs(v,u,d+1)
    def query(self, u, v):
        if self.depth[u]<self.depth[v]: u,v=v,u
        diff=self.depth[u]-self.depth[v]
        for k in range(self.LOG):
            if (diff>>k)&1: u=self.up[k][u]
        if u==v: return u
        for k in range(self.LOG-1,-1,-1):
            if self.up[k][u]!=self.up[k][v]:
                u=self.up[k][u]; v=self.up[k][v]
        return self.up[0][u]
```

21. 二叉树概念与遍历

类型	定义	关键性质
满二叉树	每个节点0或2个孩子	叶数=内部节点+1
完美二叉树	所有叶子同层	$n=2^h-1$
完全二叉树	最后一层从左连续	可用数组存；堆
BST	左<根<右	中序=升序

性质: 叶子数 n_0 =度2节点数 n_2+1 。完全BT数组: 左孩 $2i+1$, 右孩 $2i+2$, 父 $(i-1)//2$ (0起)。

遍历 (递归版)

```
def preorder(root): # 根左右
    if not root: return []
    return ([root.val]+preorder(root.left)
            +preorder(root.right))
def inorder(root): # 左根右
    if not root: return []
    return (inorder(root.left)+[root.val]
            +inorder(root.right))
def postorder(root): # 左右根
    if not root: return []
    return (postorder(root.left)
            +postorder(root.right)+[root.val])
```

层序 (BFS)

```
from collections import deque
def levelorder(root):
    if not root: return []
    dq,res=deque([root]),[]
    while dq:
        level=[]
        for _ in range(len(dq)):
            node=dq.popleft()
            level.append(node.val)
            if node.left: dq.append(node.left)
            if node.right: dq.append(node.right)
        res.append(level)
    return res
```

遍历 (迭代版)

```
def preorder_iter(root):
    if not root: return []
    stack,res=[root],[]
    while stack:
        node=stack.pop(); res.append(node.val)
        if node.right: stack.append(node.right)
        if node.left: stack.append(node.left)
    return res

def inorder_iter(root):
    stack,res,cur=[],[],root
    while cur or stack:
        while cur: stack.append(cur); cur=cur.left
        cur=stack.pop(); res.append(cur.val)
        cur=cur.right
    return res

def postorder_iter(root):
    if not root: return []
    stack,res=[root],[]
    while stack:
        node=stack.pop(); res.append(node.val)
        if node.left: stack.append(node.left)
        if node.right: stack.append(node.right)
    return res[::-1] # 根右左反转=左右根
```

建树 (已知两种遍历→唯一确定)

前序+中序建树

```
def build(preorder, inorder):
    if not preorder: return None
    root=TreeNode(preorder[0])
    idx=inorder.index(preorder[0])
    root.left=build(preorder[1:1+idx],inorder[:idx])
    root.right=build(preorder[1+idx:],inorder[idx+1:])
    return root
```

层序+中序建树

```
def build_level(levelorder, inorder):
    if not inorder: return None
    root_val=next(v for v in levelorder
                  if v in inorder)
    root=TreeNode(root_val)
    idx=inorder.index(root_val)
    left_in,right_in=inorder[:idx],inorder[idx+1:]
    left_set,right_set=set(left_in),set(right_in)
    left_lv=[v for v in levelorder if v in left_set]
    right_lv=[v for v in levelorder if v in right_set]
    root.left=build_level(left_lv,left_in)
    root.right=build_level(right_lv,right_in)
    return root
```

后序+中序建树

```
def build_post(postorder, inorder):
    if not postorder: return None
    root=TreeNode(postorder[-1])
    idx=inorder.index(postorder[-1])
    root.left=build_post(postorder[:idx],inorder[:idx])
    root.right=build_post(postorder[idx:-1],inorder[idx+1:])
    return root
```

遍历间直接转换（不建树）

```
# 前序+中序→后序
def pre_in_to_post(pre,ino):
    if not pre: return []
    root=pre[0]; idx=ino.index(root)
    L=pre_in_to_post(pre[1:1+idx],ino[:idx])
    R=pre_in_to_post(pre[1+idx:],ino[idx+1:])
    return L+R+[root]

# 后序+中序→前序
def post_in_to_pre(post,ino):
    if not post: return []
    root=post[-1]; idx=ino.index(root)
    L=post_in_to_pre(post[:idx],ino[:idx])
    R=post_in_to_pre(post[idx:-1],ino[idx+1:])
    return [root]+L+R
```

唯一性：前+中、后+中、层+中可唯一确定。前+后**不能**（除非满二叉树）。

BST由前序重建（无需中序，中序=排序后）

```
def bst_from_pre(pre):
    if not pre: return None
    root=TreeNode(pre[0])
    i=1
    while i<len(pre) and pre[i]<pre[0]: i+=1
    root.left=bst_from_pre(pre[1:i])
    root.right=bst_from_pre(pre[i:])
    return root
```

树的高度/直径

```
def height(root):
    if not root: return -1
    return 1+max(height(root.left),height(root.right))

def diameter(root):
    ans=0
    def depth(node):
        nonlocal ans
        if not node: return 0
        l,r=depth(node.left),depth(node.right)
        ans=max(ans,l+r)
        return 1+max(l,r)
    depth(root); return ans
```

AVL树（平衡二叉搜索树）

平衡因子 balance = 左子树高 - 右子树高。插入后若 |balance|>1 失衡，按四种情形旋转：

情形	条件	操作
LL	balance>1 且 新值<左孩值	右旋
LR	balance>1 且 新值>左孩值	左孩左旋，再右旋
RR	balance<-1 且 新值>右孩值	左旋
RL	balance<-1 且 新值<右孩值	右孩右旋，再左旋

```

class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
        self.height = 1

class AVL:
    def __init__(self): self.root = None
    def insert(self, value): self.root = self._insert(value, self.root)
    def _insert(self, value, node):
        if not node: return Node(value)
        if value < node.value:
            node.left = self._insert(value, node.left)
        else:
            node.right = self._insert(value, node.right)
        node.height = 1 + max(self.h(node.left), self.h(node.right))
        balance = self.h(node.left) - self.h(node.right)
        if balance > 1: # 左重
            if value < node.left.value: # LL
                return self.rotate_right(node)
            node.left = self.rotate_left(node.left) # LR
            return self.rotate_right(node)
        if balance < -1: # 右重
            if value > node.right.value: # RR
                return self.rotate_left(node)
            node.right = self.rotate_right(node.right) # RL
            return self.rotate_left(node)
        return node
    def h(self, node): return node.height if node else 0
    def rotate_left(self, z):
        y = z.right; T2 = y.left
        y.left = z; z.right = T2
        z.height = 1 + max(self.h(z.left), self.h(z.right))
        y.height = 1 + max(self.h(y.left), self.h(y.right))
        return y
    def rotate_right(self, y):
        x = y.left; T2 = x.right
        x.right = y; y.left = T2
        y.height = 1 + max(self.h(y.left), self.h(y.right))
        x.height = 1 + max(self.h(x.left), self.h(x.right))
        return x

```

22. Trie 字典树

用嵌套dict实现：每个节点是dict，键=字符，值=子节点； '#' 标记单词结尾。insert/search/starts_with均O(L)，L为单词长度。

基础模板

```

class Trie:
    def __init__(self):
        self.root = {}
    def insert(self, word):
        node = self.root
        for c in word:
            if c not in node: node[c] = {}
            node = node[c]
        node['#'] = True
    def search(self, word):
        node = self.root
        for c in word:
            if c not in node: return False
            node = node[c]
        return '#' in node
    def starts_with(self, prefix):
        node = self.root
        for c in prefix:
            if c not in node: return False
            node = node[c]
        return True

```

应用：电话号码一致性 (OJ04089)

题意：给定若干电话号码，判断是否存在一个号码是另一个号码的前缀。例如 911 和 91125426，911 是 91125426 的前缀，输出 NO（不一致）；若没有这种前缀关系则输出 YES。

思路：按长度排序后依次插入Trie。插入号码num时：- 若路径上某节点已有 '#' 标记 → 说明之前有更短的号码是num的前缀 → 不一致 - 若num插完后当前节点已有子节点 → 说明num是之前某号码的前缀 → 不一致


```
def is_consistent(numbers):
    numbers.sort(key=len)          # 短的先插入
    root = {}
    for num in numbers:
        node = root
        for c in num:
            if '#' in node:        # 路径上已有完整号码→它是当前号码前缀
                return False
            if c not in node:
                node[c] = {}
            node = node[c]
        if node:                   # 当前号码插完，节点还有子树→它是别人的前缀
            return False
        node['#'] = True
    return True

# 示例
print(is_consistent(["911", "97625999", "91125426"]))    # False
print(is_consistent(["113", "12340", "123440", "12345", "98346"]))    # True
```

完整OJ提交版（多组测试数据）

```
import sys
def solve():
    data = sys.stdin.read().split()
    idx = 0
    T = int(data[idx]); idx += 1
    for _ in range(T):
        n = int(data[idx]); idx += 1
        numbers = []
        for _ in range(n):
            numbers.append(data[idx]); idx += 1
        numbers.sort(key=len)
        root = {}
        ok = True
        for num in numbers:
            node = root
            for c in num:
                if '#' in node:
                    ok = False; break
                if c not in node: node[c] = {}
                node = node[c]
            if not ok: break
            if node:
                ok = False; break
            node['#'] = True
        print("YES" if ok else "NO")
solve()
```

其他典型应用：自动补全、最长公共前缀、前缀出现次数统计(节点加count)。

0-1 Trie: 求数组中两数异或最大值

把整数按二进制位（高位到低位）插入Trie。查询某数的最大异或时，每位贪心走**相反**的分支（让异或结果该位为1）。

```

class BinTrie:
    def __init__(self):
        self.root = [None, None] # children[0], children[1]
    def insert(self, num, BITS=30):
        node = self.root
        for i in range(BITS, -1, -1):
            bit = (num >> i) & 1
            if not node[bit]:
                node[bit] = [None, None]
            node = node[bit]
    def max_xor(self, num, BITS=30):
        node = self.root
        res = 0
        for i in range(BITS, -1, -1):
            bit = (num >> i) & 1
            want = 1 - bit # 想走相反位
            if node[want]:
                res |= (1 << i)
                node = node[want]
            else:
                node = node[bit]
        return res

# 求数组任意两数最大异或
def find_max_xor(nums):
    trie = BinTrie()
    ans = 0
    for x in nums:
        trie.insert(x)
    ans = max(ans, trie.max_xor(x))
    return ans

```

23. KMP算法

构造LPS数组 O(N)

```

def build_lps(s):
    n=len(s); lps=[0]*n; length=0; i=1
    while i<n:
        if s[i]==s[length]:
            length+=1; lps[i]=length; i+=1
        else:
            if length!=0: length=lps[length-1]
            else: lps[i]=0; i+=1
    return lps

```

KMP匹配

```

def kmp_search(text, pattern):
    if not pattern: return []
    lps=build_lps(pattern)
    res,i,j=[],0,0
    while i<len(text):
        if text[i]==pattern[j]:
            i+=1; j+=1
            if j==len(pattern):
                res.append(i-j)
                j=lps[j-1]
        else:
            if j!=0: j=lps[j-1]
            else: i+=1
    return res

```

最小循环节（最小周期）

设 $lps[n-1]=k$ ，若 $n\%(n-k)=0$ 则最小周期= $n-k$ ，串由 $s[n-k]$ 重复 $n/(n-k)$ 次。

```

def min_period(s):
    n=len(s); lps=build_lps(s)
    p=n-lps[n-1]
    if n%p==0: return p, n//p
    return p, None # 不完整循环

```

24. Manacher回文串

在O(n)时间内求出字符串中所有回文子串信息。插入 # 把奇偶回文统一处理， half[i] 记录以i为中心的回文半径。

```
def manacher(s):
    s1 = '#'.join('^' + s + '&') # 首尾加哨兵，字符间插#
    half_l = [0] * len(s1)
    c, r = 0, 0 # 当前回文中心c、右边界r
    for i in range(1, len(s1) - 1):
        if i < r:
            half_l[i] = min(half_l[2*c - i], r - i) # 镜像复用
        while s1[i - half_l[i] - 1] == s1[i + half_l[i] + 1]:
            half_l[i] += 1
        if i + half_l[i] > r: # 更新最右回文
            c, r = i, i + half_l[i]
    return half_l
```

half_l[i] 正好是原串中以该中心的最长回文长度
回文子串总数 = sum(h//2 + ...), 最长回文长度 = max(half_l)

判断原串区间 [a,b) 是否回文 (a,b为原串索引) :

```
def is_pal(half_l, a, b):
    return half_l[a + b + 1] ≥ (b - a) # 中心在 a+b+1 (含#后坐标)
```

只判整体回文用 deque 更简单:

```
from collections import deque
def pal(s):
    d = deque(s)
    while len(d) > 1:
        if d.popleft() ≠ d.pop(): return False
    return True
```

25. TSP 旅行商

状压DP (Held-Karp) : $dp[S][i]$ = 已访问集S、停在i的最短路。 $O(n^2 \cdot 2^n)$, $n \leq 20$ 。

```
import sys
def solve():
    data = sys.stdin.read().split(); it = iter(data)
    n = int(next(it))
    dist = [[int(next(it)) for _ in range(n)] for _ in range(n)]
    size = 1 << n; INF = float('inf')
    dp = [[INF] * n for _ in range(size)]
    dp[1][0] = 0
    for i in range(1, size, 2): # 奇数mask (第0位=1)
        for j in range(n):
            curr = dp[i][j]
            if curr == INF: continue
            for k in range(1, n): # 中间不回0
                if (i >> k) & 1: continue
                new_cost = curr + dist[j][k]
                if new_cost < dp[i | (1 << k)][k]:
                    dp[i | (1 << k)][k] = new_cost
    ans = INF; full = size - 1
    for i in range(1, n):
        c = dp[full][i] + dist[i][0]
        if c < ans: ans = c
    print(ans)
solve()
```

优化要点: ①快速IO; ②list代替dict; ③奇数步长省一半; ④手动比较替代min()。

变形: 路径TSP(不回起点) $\rightarrow \min(dp[full][i])$; TSP计数 $\rightarrow \min$ 改sum。

26. 经典题: 招生工作

OJ28749: n学生颜色R/P/W, m志愿者各负责一组。沟通使学生颜色循环+1($R \rightarrow P \rightarrow W \rightarrow R$)。求最少沟通次数使全部变R。

建模: $R=0, P=1, W=2$ 。志愿者为节点, 学生为边(只有一个志愿者时连虚拟节点0)。方程: $x_u + x_v \equiv (3 - \text{init}) \pmod 3$ 。

求解: 逐连通分量BFS。含节点0的固定 $x_0=0$ 唯一确定; 不含0的枚举 $\text{root} \in \{0, 1, 2\}$ 取最小。

```

import sys
from collections import deque

def solve():
    data=sys.stdin.buffer.read().split()
    if not data: return
    it=iter(data); n=int(next(it)); m=int(next(it))
    color_str=next(it).decode()
    init=[0]*(n+1)
    for i,ch in enumerate(color_str,1):
        init[i]={'R':0,'P':1,'W':2}[ch]
    stu_vol=[[[] for _ in range(n+1)]
    for vol in range(1,m+1):
        k=int(next(it))
        for _ in range(k): stu_vol[int(next(it))].append(vol)
    for stu in range(1,n+1):
        if not stu_vol[stu] and init[stu]!=0:
            print("impossible"); return
    graph=[[[] for _ in range(m+1)]
    for stu in range(1,n+1):
        lst=stu_vol[stu]
        if not lst: continue
        u,v=(lst[0],0) if len(lst)==1 else (lst[0],lst[1])
        target=(3-init[stu])%3
        graph[u].append((v,target)); graph[v].append((u,target))
    visited=[False]*(m+1); total=0
    # 含虚拟节点0的分量
    if graph[0]:
        comp=[]; q=deque([0]); visited[0]=True
        while q:
            u=q.popleft(); comp.append(u)
            for v,_ in graph[u]:
                if not visited[v]: visited[v]=True; q.append(v)
        assign={0:0}; q=deque([0]); ok=True
        while q and ok:
            u=q.popleft()
            for v,target in graph[u]:
                val_v=(target-assign[u])%3
                if v not in assign: assign[v]=val_v; q.append(v)
                elif assign[v]!=val_v: ok=False; break
            if not ok: print("impossible"); return
        total+=sum(assign[nd] for nd in comp if nd!=0)
    # 不含0的分量
    for start in range(1,m+1):
        if visited[start]: continue
        comp=[]; q=deque([start]); visited[start]=True; comp.append(start)
        while q:
            u=q.popleft()
            for v,_ in graph[u]:
                if v!=0 and not visited[v]:
                    visited[v]=True; q.append(v); comp.append(v)
        best=None
        for rv in range(3):
            assign={start:rv}; q=deque([start]); ok=True
            while q and ok:
                u=q.popleft()
                for v,target in graph[u]:
                    if v==0: continue
                    val_v=(target-assign[u])%3
                    if v not in assign: assign[v]=val_v; q.append(v)
                    elif assign[v]!=val_v: ok=False; break
            if ok:
                cost=sum(assign[nd] for nd in comp)
                if best is None or cost<best: best=cost
        if best is None: print("impossible"); return
        total+=best
    print(total)
solve()

```

27. 经典回溯：骑士周游/八皇后

骑士周游 (Knight's Tour)

题意：在 $n \times n$ 棋盘上，骑士（马）从指定起点出发，按"日"字走法（8个方向），要求恰好访问每个格子一次。输出 success/fail。

Warnsdorff启发式：每步贪心选"下一步可达未访问格子数最少"的邻居。直觉：先走"死角"，把"活路"留到后面。对大多数起点可以**无回溯**一次走通，时间 $O(n^2)$ 。

```

n = int(input())
x, y = map(int, input().split())
x -= 1; y -= 1

dirs = [(2,1),(1,2),(-1,2),(-2,1),(-2,-1),(-1,-2),(1,-2),(2,-1)]
visited = [[False]*n for _ in range(n)]
cx, cy = x, y
visited[cx][cy] = True
count = 1

while count < n * n:
    next_steps = []
    for dx, dy in dirs:
        nx, ny = cx+dx, cy+dy
        if 0 ≤ nx < n and 0 ≤ ny < n and not visited[nx][ny]:
            # Warnsdorff: 计算(nx,ny)的未访问邻居数
            cnt = 0
            for dx2, dy2 in dirs:
                nnx, nny = nx+dx2, ny+dy2
                if 0 ≤ nnx < n and 0 ≤ nny < n and not visited[nnx][nny]:
                    cnt += 1
            next_steps.append((cnt, nx, ny))
    if not next_steps:
        break
    next_steps.sort()           # 贪心: 邻居最少的优先
    _, cx, cy = next_steps[0]
    visited[cx][cy] = True
    count += 1

print("success" if count == n*n else "fail")

```

注意: Warnsdorff是启发式，极少数情况可能fail。如需保证正确可加回溯:

```

def solve(cx, cy, count):
    if count == n * n:
        return True
    next_steps = []
    for dx, dy in dirs:
        nx, ny = cx+dx, cy+dy
        if 0 ≤ nx < n and 0 ≤ ny < n and not visited[nx][ny]:
            cnt = sum(1 for dx2, dy2 in dirs
                      if 0 ≤ nx+dx2 < n and 0 ≤ ny+dy2 < n
                      and not visited[nx+dx2][ny+dy2])
            next_steps.append((cnt, nx, ny))
    next_steps.sort()
    for _, nx, ny in next_steps:
        visited[nx][ny] = True
        if solve(nx, ny, count+1):
            return True
        visited[nx][ny] = False    # 回溯
    return False

```

八皇后 (N-Queens)

题意: 在 $n \times n$ 棋盘上放 n 个皇后，使得任意两个皇后不在同一行、同一列、同一对角线上。输出所有解的数量，或输出所有解的排列。

思路: 逐行放置，用三个集合记录已被占用的列、主对角线($\text{row} - \text{col}$)、副对角线($\text{row} + \text{col}$)。

```
def n_queens(n):
    solutions = []
    queens = [0] * n      # queens[row] = col

    def backtrack(row, cols, diag1, diag2):
        if row == n:
            solutions.append(queens[:])
            return
        for col in range(n):
            if col in cols or (row-col) in diag1 or (row+col) in diag2:
                continue
            queens[row] = col
            cols.add(col)
            diag1.add(row - col)
            diag2.add(row + col)
            backtrack(row + 1, cols, diag1, diag2)
            cols.remove(col)
            diag1.remove(row - col)
            diag2.remove(row + col)

    backtrack(0, set(), set(), set())
    return solutions

# 输出所有解
n = int(input())
sols = n_queens(n)
print(len(sols))          # 解的数量 (n=8时为92)
for s in sols:
    print(' '.join(str(c+1) for c in s)) # 每行皇后所在列号(1-based)
```

位运算优化版 (n≤15时显著加速, 只求解数) :

用整数的二进制位表示被占用的列/对角线, 用 **lowbit** 枚举可用位置, 避免集合操作。

```
def count_queens(n):
    count = 0
    full = (1 << n) - 1      # n位全1

    def dfs(row, cols, diag1, diag2):
        nonlocal count
        if row == n:
            count += 1
            return
        # 可用位置 = 全集 & ~(已占用)
        avail = full & ~(cols | diag1 | diag2)
        while avail:
            pos = avail & (-avail) # lowbit: 取最低位的1
            avail -= pos
            dfs(row + 1,
                cols | pos,
                (diag1 | pos) << 1, # 主对角线: 下一行左移1位
                (diag2 | pos) >> 1) # 副对角线: 下一行右移1位

    dfs(0, 0, 0, 0)
    return count

print(count_queens(int(input()))) # n=8 → 92, n=12 → 14200
```

对角线编码要点: - 主对角线 \ : 同一对角线上 row-col 相同 (集合版) ; 位运算版每下一行整体左移1位 - 副对角线 / : 同一对角线上 row+col 相同 (集合版) ; 位运算版每下一行整体右移1位

28. 数值格式化

浮点格式化

```
x = 3.14159265
f"{x:.2f}"          # 3.14
f"{x:10.2f}"        # '      3.14'
f"{x:<10.2f}"        # '3.14      '
f"{x:010.2f}"        # 0000003.14
f"{x:+.2f}"          # +3.14
f"{x:.2e}"           # 3.14e+00
f"{x:.2%}"           # 314.16%
# round银行家舍入: round(2.5)=2, round(3.5)=4
```

整数格式化

```
f"{42:05d}"          # 00042
f"{42:b}"             # 101010
f"{42:#x}"            # 0x2a
f"{1234567:,}"        # 1,234,567
```

浮点陷阱

```
0.1+0.2==0.3          # False!
abs(0.1+0.2-0.3)<1e-9 # 正确判等
```

整除取模

```
17//5      # 3    向下取整
-17//5      # -4   注意! 不是-3
17%5        # 2
-17%5        # 3    注意! 不是-2
pow(2,10,1000) # 24 快速幂取模
```

向上取整

```
import math
math.ceil(17/5)      # 4
(17+5-1)//5          # 4  整数写法
```

类型转换

```
int("ff",16)      # 255
int(3.9)           # 3  向0截断
float("inf")        # 无穷大
```

29. Python考试陷阱

字符串: "12"<"2" 为True (字典序) , 数值比较先 int() 。 split() 按任意空白, split(' ') 只按单空格。

递归: 默认深度~1000, 大数据用 sys.setrecursionlimit(10**6) 或改迭代。

列表: [[0]*n]*m **错!** 正确: [[0]*n for _ in range(m)] 。 pop(0) 是O(n), 用 deque。

IO优化:

```
import sys
data=sys.stdin.buffer.read().split()
it=iter(data); n=int(next(it))
```

循环内频繁print拖慢一两个数量级。

浮点: abs(a-b)<1e-9 判等, 不用 == 。

集合: in set O(1), in list O(n)。

排序: sorted(d.items(),key=lambda kv:kv[1]) 。 Python的sort是**稳定**的。

闭包: 循环中lambda捕获变量按引用, 用 lambda i=i: ... 固定。

常用模块速查

模块	用途
heapq	小顶堆 (大顶取负)
bisect	有序list二分
lru_cache	记忆化搜索
Counter	计数
defaultdict	默认字典
deque	双端队列

数据规模选算法: n≤20暴力, n≤1000 O(n²), n≤10⁵ O(nlogn)。

大顶堆: Python的heapq是小顶堆, 大顶堆有两种做法:

方法一: 取负数 (最常用, 适合纯数值)

```
import heapq

heap = []
# 入堆: 取负
for x in [3, 1, 4, 1, 5, 9, 2, 6]:
    heapq.heappush(heap, -x)

# 出堆: 取负还原
print(-heapq.heappop(heap)) # 9
print(-heapq.heappop(heap)) # 6

# 查看堆顶 (不弹出)
print(-heap[0])            # 5

# heapify 也可以
arr = [3, 1, 4, 1, 5]
max_heap = [-x for x in arr]
heapq.heapify(max_heap)
print(-heapq.heappop(max_heap)) # 5
```

方法二: 重载 __lt__ (适合元组/对象)

```
import heapq

class MaxItem:
    def __init__(self, val):
        self.val = val
    def __lt__(self, other):
        return self.val > other.val # 反向比较

heap = []
for x in [3, 1, 4, 1, 5, 9]:
    heapq.heappush(heap, MaxItem(x))
print(heapq.heappop(heap).val) # 9

# 带优先级的任务队列 (按优先级从大到小)
tasks = []
heapq.heappush(tasks, MaxItem((3, "low")))
heapq.heappush(tasks, MaxItem((10, "high")))
print(heapq.heappop(tasks).val) # (10, 'high')
```

常用堆操作速查

操作	写法	复杂度
建堆	heapq.heapify(list)	O(n)
入堆	heapq.heappush(heap, x)	O(logn)
弹出最小	heapq.heappop(heap)	O(logn)
查看堆顶	heap[0]	O(1)
入堆+弹出	heapq.heappushpop(heap, x)	O(logn)
前k小	heapq.nsmallest(k, iterable)	O(n-logk)
前k大	heapq.nlargest(k, iterable)	O(n-logk)

30. 链表

```
class ListNode:
    def __init__(self, x):
        self.val = x
        self.next = None
```

快慢指针：找中点 / 判环

```
# 找中点（偶数长度取右中点）
def middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow

# 判断是否有环
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast: return True
    return False
```

找入环节点：相遇后把re放头部，与slow同步走，相遇点即入环点（推导：a=n-c-(相遇点环内偏移)）。

```
def detect_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            re = head
            while re is not slow:
                re = re.next; slow = slow.next
            return re
    return None
```

反转链表

```
def reverse(head):
    prev = None
    while head:
        nxt = head.next
        head.next = prev
        prev = head
        head = nxt
    return prev
```

哨兵节点：合并两个有序链表

```
def merge(l1, l2):
    dummy = h = ListNode(0) # 哨兵
    while l1 and l2:
        if l1.val <= l2.val:
            h.next = l1; l1 = l1.next
        else:
            h.next = l2; l2 = l2.next
        h = h.next
    h.next = l1 or l2
    return dummy.next
```

两两交换节点

```
def swap_pairs(head):
    dummy = a = ListNode(0); a.next = head
    while a.next and a.next.next:
        first = a.next
        a.next = first.next
        first.next = a.next.next
        a.next.next = first
        a = first
    return dummy.next
```

31. 树形DP

宝藏二叉树（OJ24637，相邻节点不能同时取，完全二叉树用数组）

```
N = int(input())
t = list(map(int, input().split()))
dp1 = [0]*N # 取当前节点
dp2 = [0]*N # 不取当前节点
for i in range(N-1, -1, -1):
    if 2*i+2 < N:
        dp1[i] = t[i] + dp2[2*i+1] + dp2[2*i+2]
        dp2[i] = max(dp1[2*i+1], dp2[2*i+1]) + max(dp1[2*i+2], dp2[2*i+2])
    elif 2*i+1 < N:
        dp1[i] = t[i] + dp2[2*i+1]
        dp2[i] = dp1[2*i+1]
    else:
        dp1[i] = t[i]
print(max(dp1[0], dp2[0]))
```

多叉树树形DP（按深度从深到浅，先算子节点）：dp1取当前节点，dp2不取。c1 += max(0, dp2[child])（取当前则子节点不能取），c2 += max(0, dp1[child], dp2[child])。


```

n = int(input())
r = [0] + [int(input()) for _ in range(n)]
graph = [[] for _ in range(n+1)]
father = [0]*(n+1)
for _ in range(n-1):
    l, k = map(int, input().split())
    father[l] = k; graph[k].append(l)
root = next(t for t in range(1, n+1) if father[t] == 0)
# 按深度分层
depth = []
stack = [(root, 0)]
while stack:
    node, d = stack.pop()
    if len(depth) <= d: depth.append([])
    depth[d].append(node)
    for ch in graph[node]: stack.append((ch, d+1))
dp1 = [0]*(n+1); dp2 = [0]*(n+1)
for nodes in reversed(depth): # 从深到浅
    for node in nodes:
        c1 = c2 = 0
        for ch in graph[node]:
            c1 += max(0, dp2[ch])
            c2 += max(0, dp1[ch], dp2[ch])
        dp1[node] = r[node] + c1
        dp2[node] = c2
print(max(dp1[root], dp2[root]))

```

32. 括号嵌套树

用括号表示节点关系的字符串。二叉树用 * 表示空节点（遍历时去掉）。**核心：用栈，遇) 弹栈合并子树。**

二叉树（已知子节点数，直接pop）OJ27637

```

# 输入如 A(B(*,C),D)
pre, stack = [], []
for ch in input():
    if ch == ')': # 二叉树固定结构
        r = stack.pop(); stack.pop() # 右,逗号
        l = stack.pop(); stack.pop() # 左,左括号
        root = stack.pop()
        stack.append(l + root + r) # 拼中序
    else:
        stack.append(ch)
        if ch not in '(', ':':
            pre.append(ch)
pres = ''.join(pre).replace('*', '') # 前序
mid = ''.join(stack).replace('*', '') # 中序
print(pres); print(mid)

```

多叉树（不知子节点数，while循环）OJ24729

```

# 输出前序 + 后序
pre, stack = [], []
for ch in input():
    if ch == ')':
        t = ''
        while stack: # 多叉树不定长
            re = stack.pop()
            if re == '(':
                t = t + stack.pop() # 父节点
                break
            if re != ',':
                t = re + t # 累积子节点(后序)
        stack.append(t)
    elif ch != ',':
        stack.append(ch)
    if ch != '(':
        pre.append(ch)
print(''.join(pre)) # 前序
print(''.join(stack)) # 后序

```

前序 = 字符串中节点值出现的顺序。中序只有二叉树能求。后序多叉/二叉都可。